울산대학교 공과대학 IT융합학부
IT융합전공

RNN (Recurrent Neural Network)

- Basics and understanding -

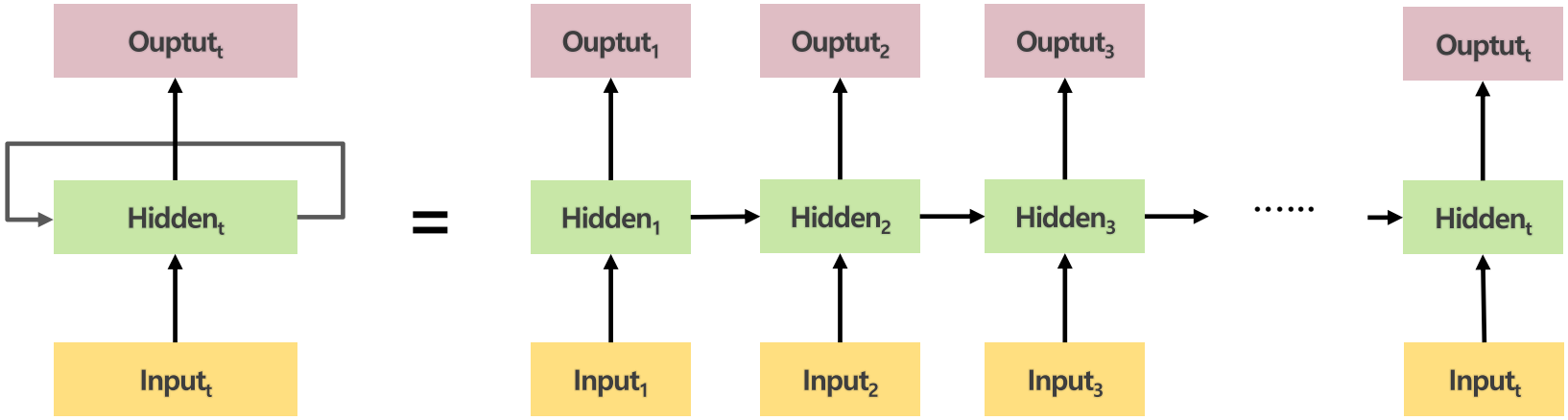
Fall 2025

School of IT Convergence

Prof. Dongsup Jin

Review - RNN

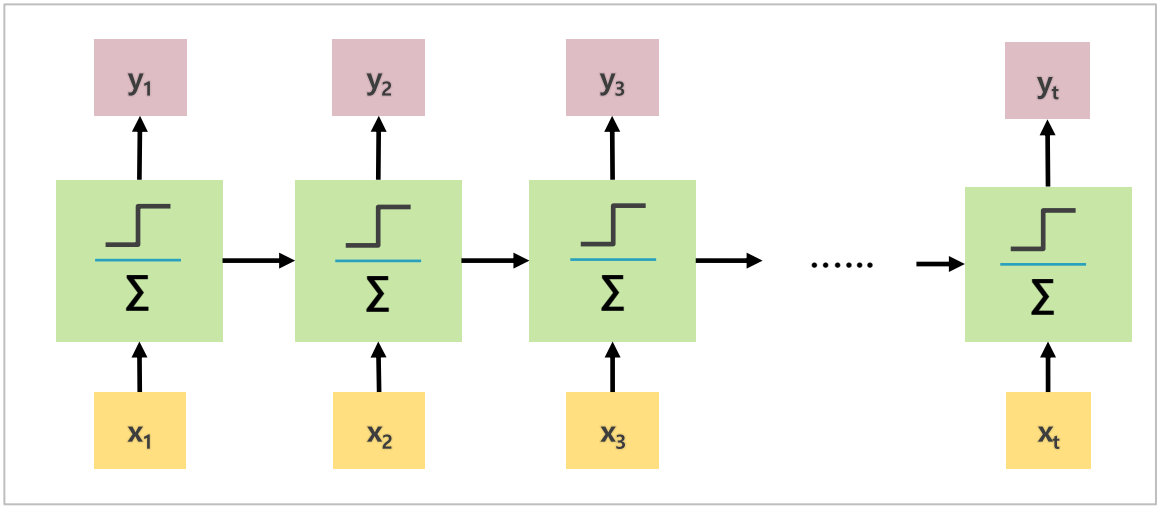
- RNN (Recurrent Neural Network)
  - Train sequential data or time series data
  - Create the current hidden vector from the previous hidden vector (t-1) and current input vector (t)

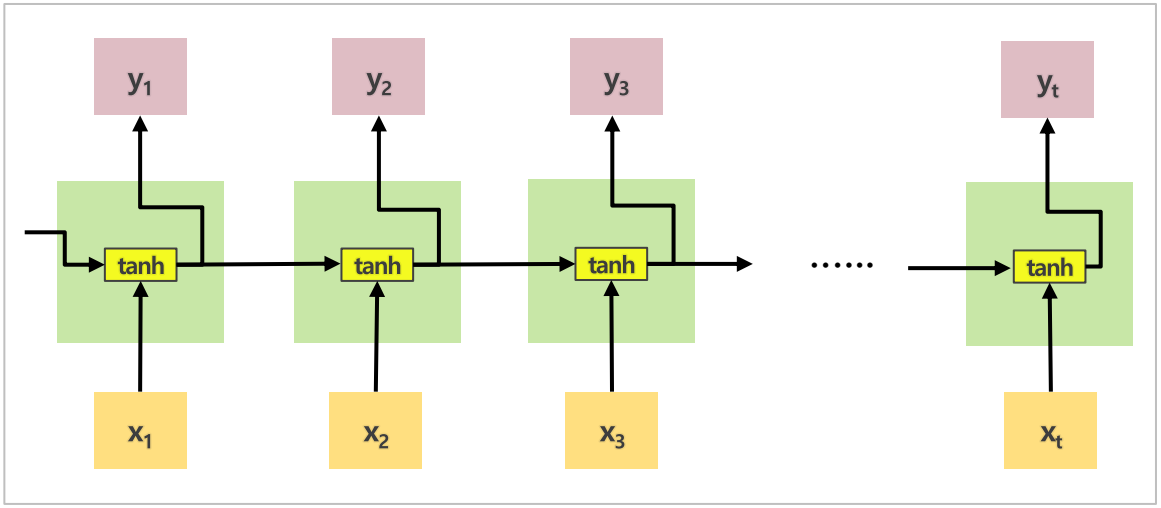


N:N RNN structure

RNN (1)

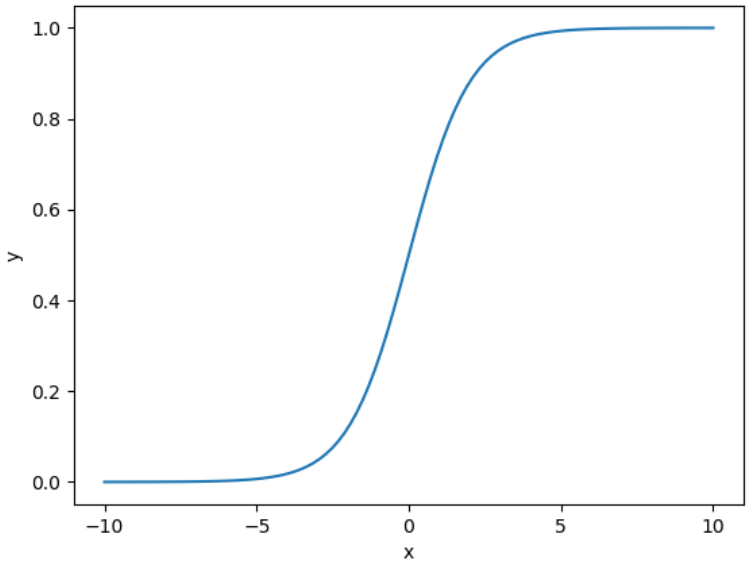
- RNN's structure

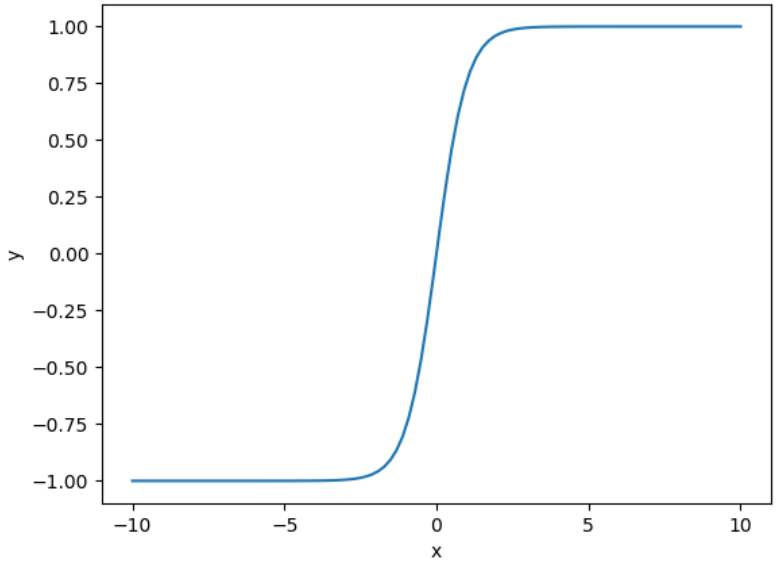




RNN (2)

- **Activation functions (Sigmoid vs Tanh)**





$$S(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{e^x+1} = 1 - S(-x)$$

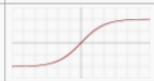
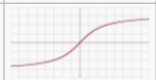
Sigmoid

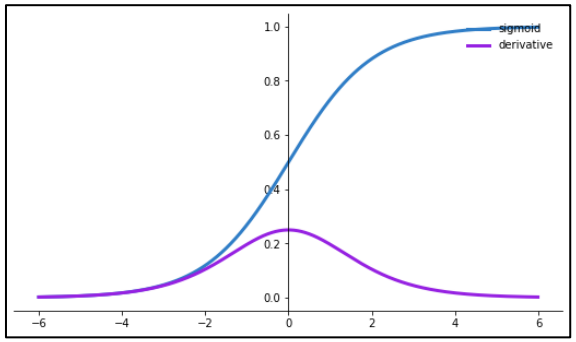
$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanh

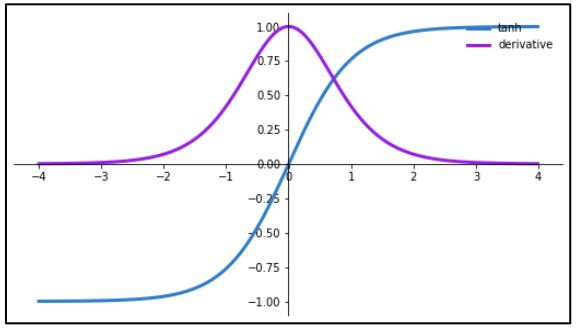
RNN (3)

- **Typical activation functions and their derivatives**

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1+e^{-x}}$	$f'(x) = f(x)(1-f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2+1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1+e^x)$	$f'(x) = \frac{1}{1+e^{-x}}$



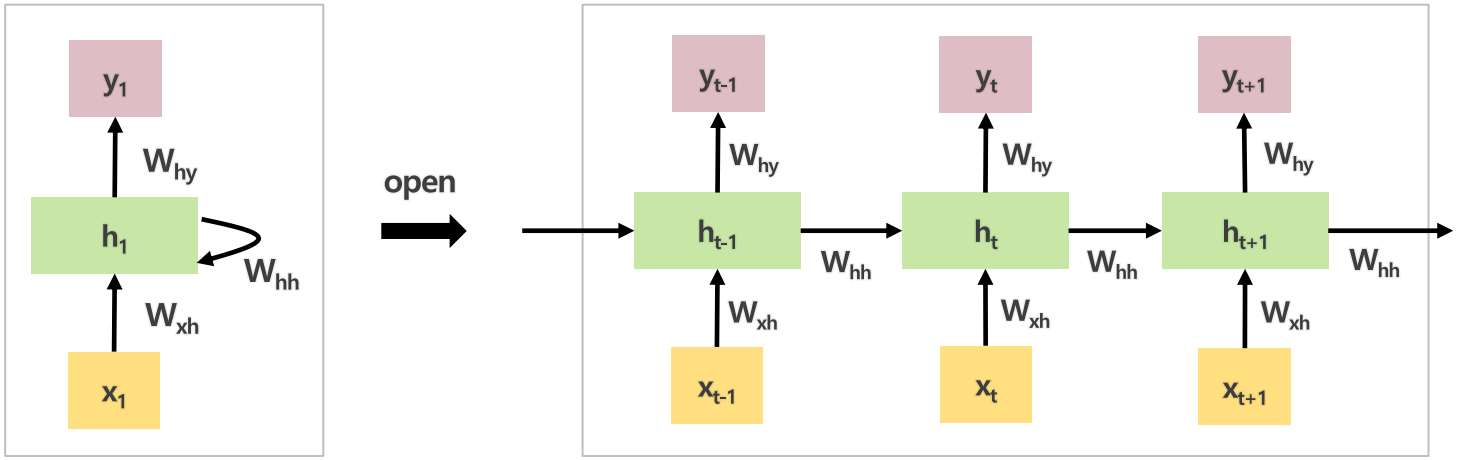
Sigmoid and its derivative



Tanh and its derivative

RNN (4)

- RNN's structure
  - W_{xh}: weights transferred from the input layer to the hidden layer
  - W_{hh}: weights transferred from the hidden layer at time t to the hidden layer at time t+1
  - W_{hy}: weights transferred from the hidden layer to the output layer



RNN (5)

- RNN's structure

1. Hidden layer
   - Hyperbolic tangent function

$$W_{xh} = \tanh(h_t)$$

$$h_t = W_{hh} * h_{t-1} + W_{xh} * x_t$$

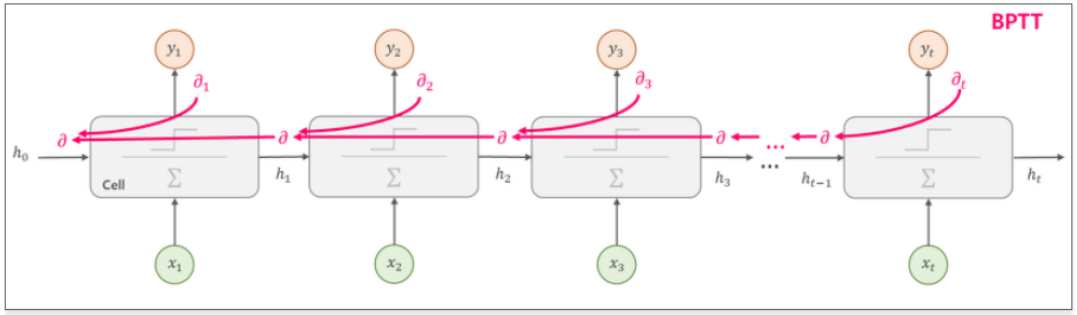
No bias case !!

2. Output layer
   - Softmax function

$$y_t = \text{softmax}(W_{hy} * h_t)$$

3. Error
   - Error between real-value and predicted value for each step

4. BPTT (Back-propagation through time)
   - Measure errors at each step and pass it to the previous step
   - Update weights W_{xh}, W_{hh}, W_{hy}

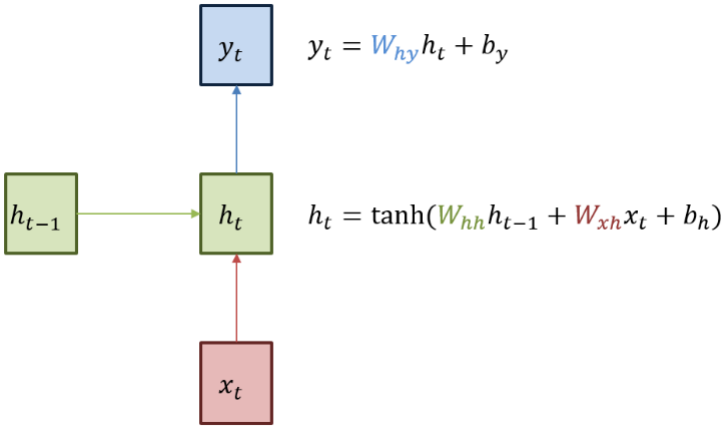


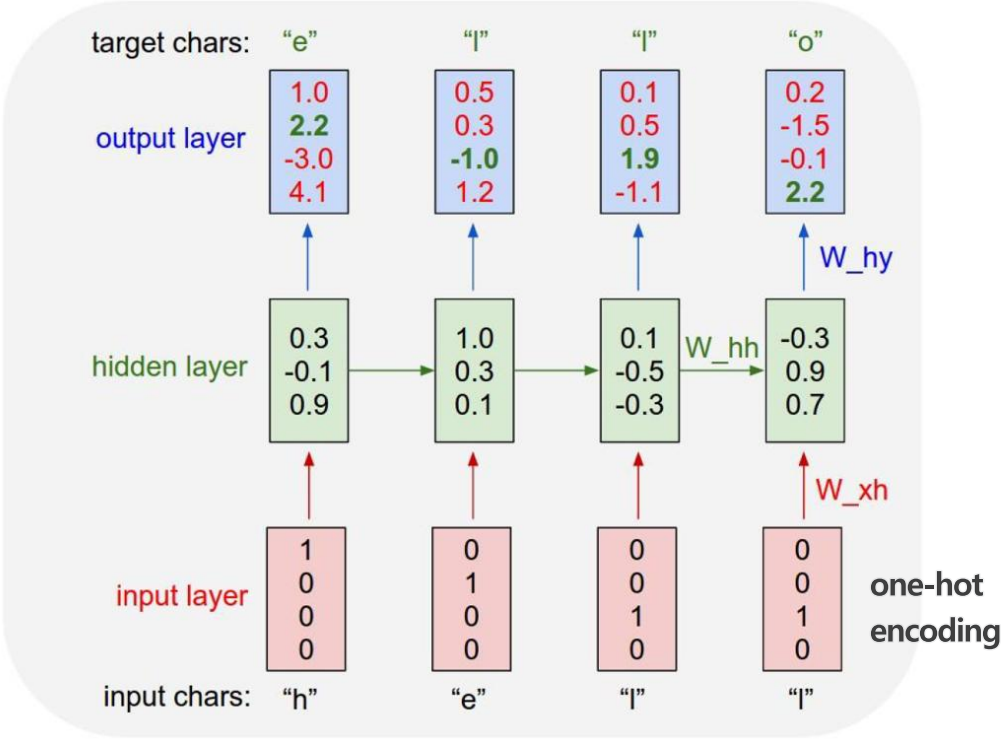
Vanishing gradient problem

➔ Truncated BPTT

RNN (6)

- **RNN's structure**

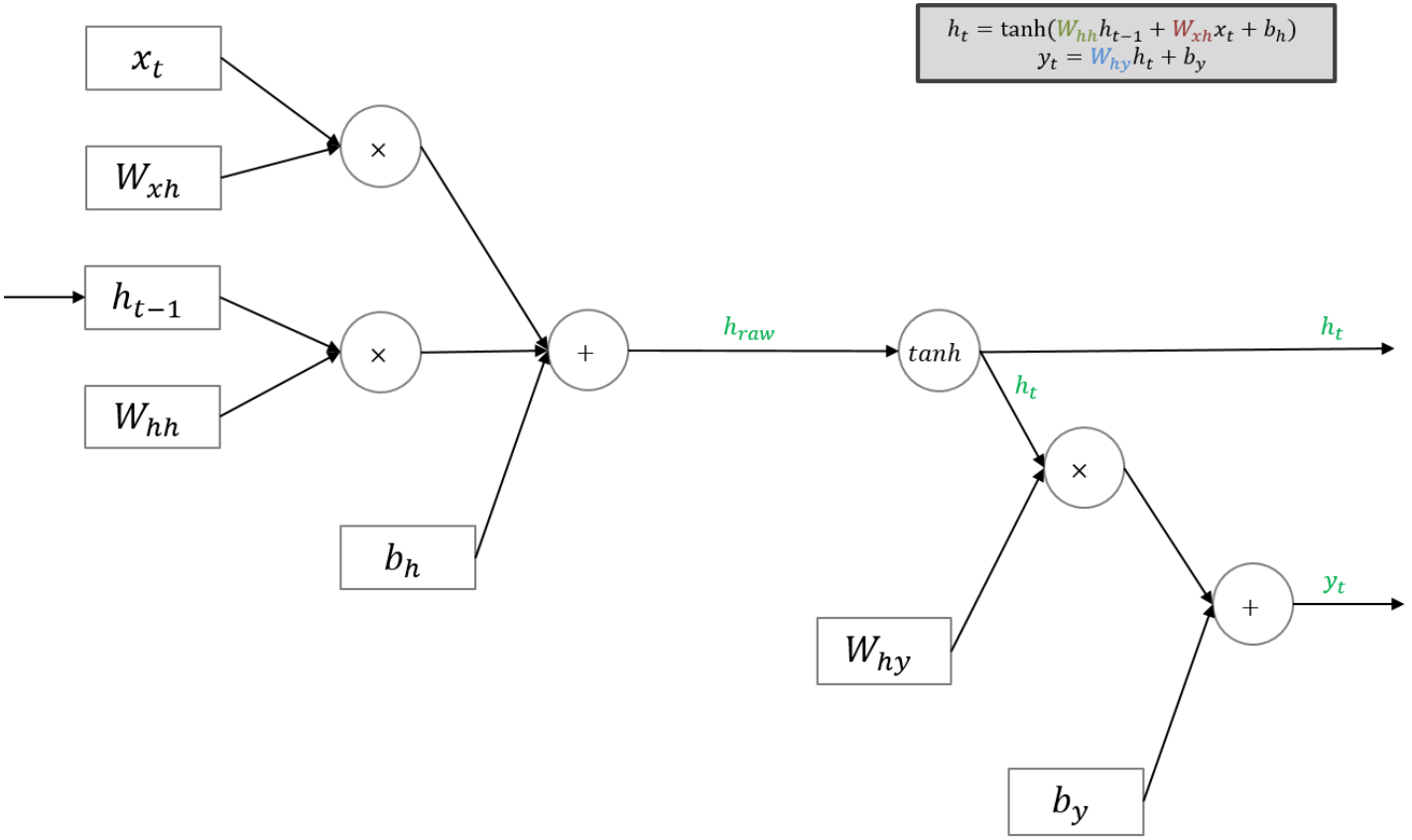




'hello' example

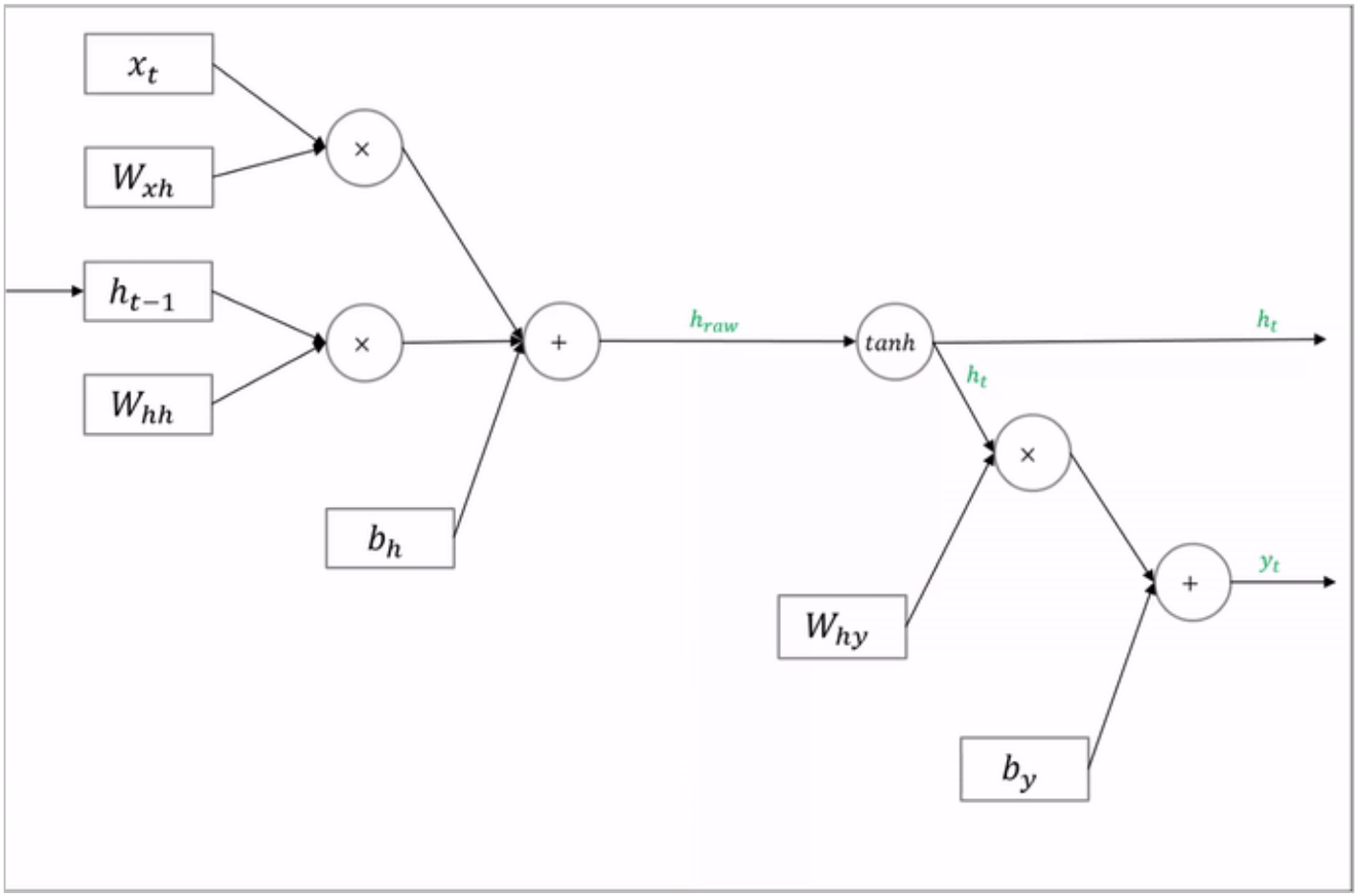
RNN (7)

- RNN propagation



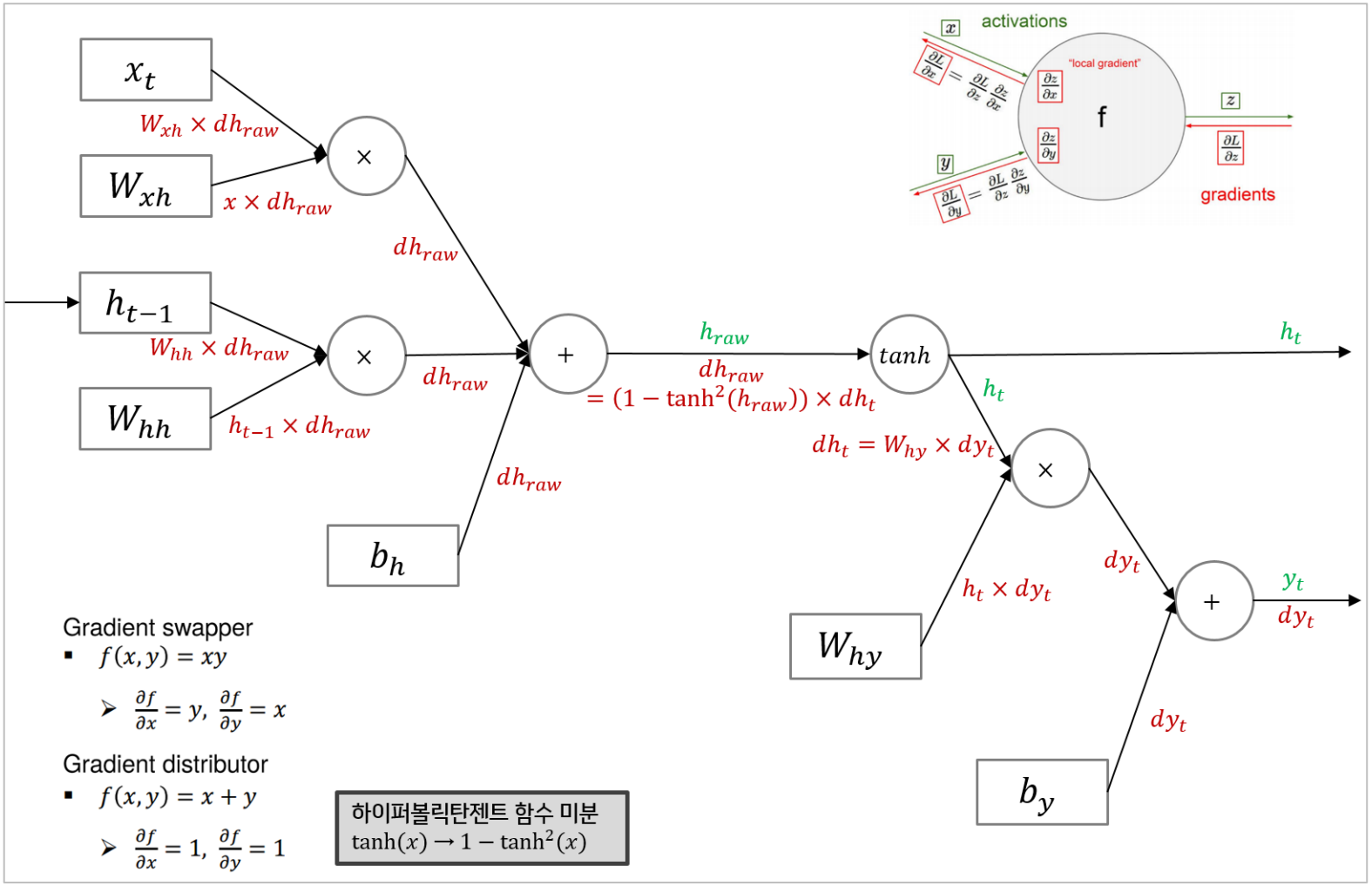
RNN (7)

- RNN back-propagation



RNN (8)

- RNN back-propagation



Q&A